Problem 1: End-to-End Speech-to-Intent for Indic Calls - Plan

Prepared for Prashant Gagneja | 13 Oct 2026 | Assignment from Meenakshi Ravisankar (Samvaad, Sarvam AI)

Path B builds one model that maps 8 kHz Indic call audio straight to one of about 10 intents or an other outcome, with a reject gate against false triggers. It finishes in 7 working days because data collection is done beforehand and is not counted. A cascaded ASR baseline is built first, so every result is compared with the simpler alternative.

The task, the scope and the 7-day clock

The assignment is to route noisy, code-mixed Indic speech directly to a fixed list of intents, minimising false triggers and handling out-of-vocabulary entities in real time. Path B reads "directly" as audio in, intent out, with no ASR transcript in the decision path.

Phrase in the problem	What it means for Path B	How to measure it
noisy, code-mixed Indic speech	Train and test on 8 kHz audio in Hinglish and one more language	Accuracy per noise level and per language
fixed list of intents	About 10 intents plus an "other" class on the model's output	In-scope accuracy
minimising false triggers	A reject gate on top of the intent head, evaluated per type of unknown input	False-accept rate at fixed in-scope recall
out-of-vocabulary entities	An auxiliary transcript decoder feeds a phonetic gazetteer match; unmatched spans are flagged	Entity accuracy and unknown-flag rate
real time	One forward pass per utterance on a CPU-friendly encoder	Real-time factor and milliseconds from end of speech to decision

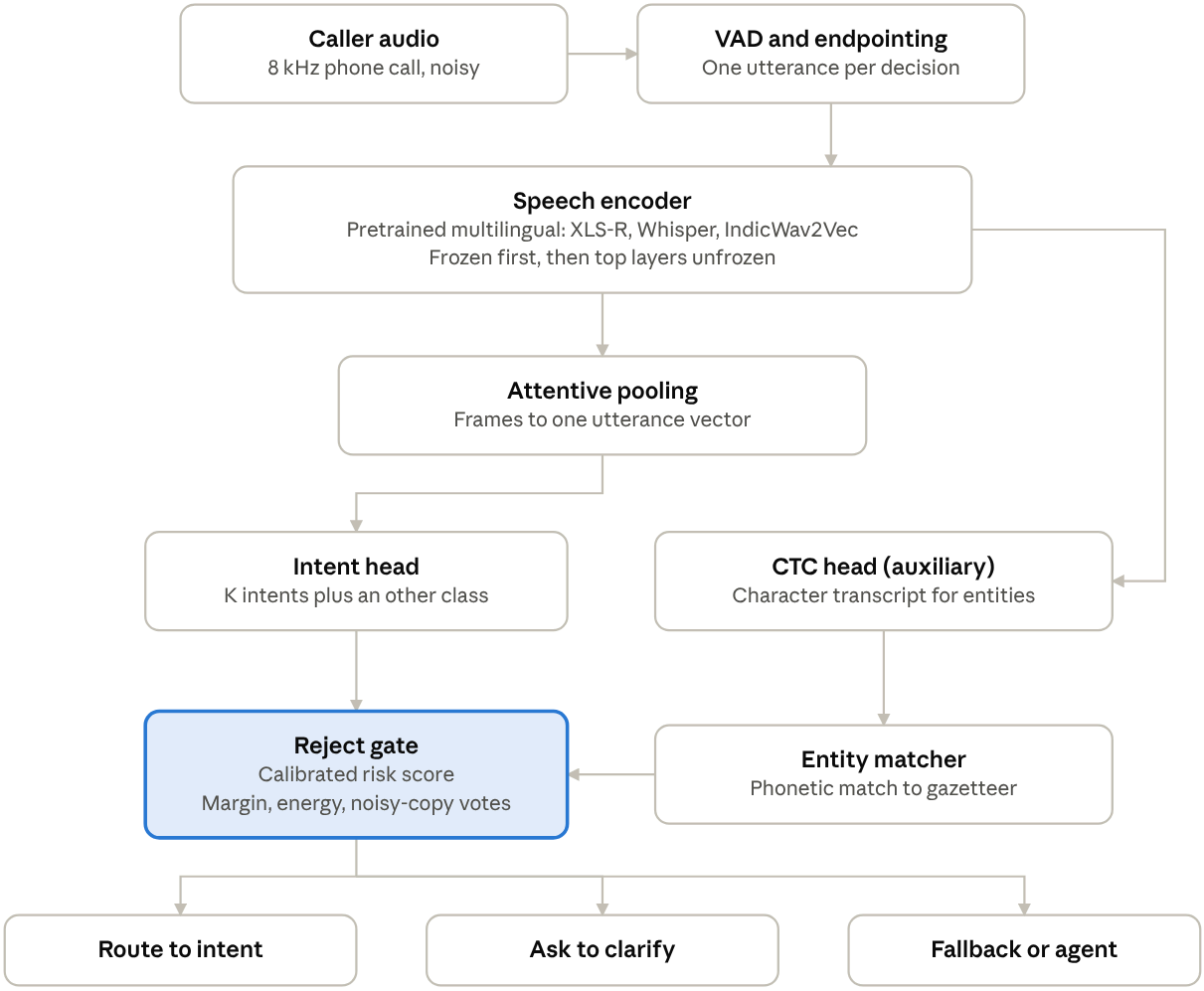
Ground rules for the 7 days

- **The clock:** 7 days cover modelling, evaluation and writing. Data collection happens before Day 1 and is not counted, but it has a ready gate (see the data section). If the gate fails, Day 1 does not start.
- **Baseline first:** the cascaded baseline (Saaras transcript, then a text intent classifier) is built on Day 1 and every later result is compared with it.
- **Evidence over architecture:** every added component must justify itself in an ablation, and failures are reported.
- **Scoped claims:** results appear in three separate tables (synthetic, real speakers, public benchmark) that are never merged, and nothing is claimed about natural Indic telephony beyond the real-speaker set.
- **Scope assumption:** Problem 1 alone. The scope and deadline questions are with Meenakshi; until she replies, the plan assumes Problem 1 only and a 7-day core.

What end-to-end buys, and what it costs. It removes the ASR error cascade, needs one model, lowers latency and optimises directly for the intent task. In return it needs audio training data, gives no transcript to inspect when it fails, makes unseen entities harder, and risks learning TTS artifacts. The sections below answer each cost, and the baseline shows whether the added complexity pays off.

Architecture

Audio goes straight to an intent, and a reject gate decides whether to act



Highlighted: the reject gate. Training only: a text classifier teaches the intent head.

end-to-end path · 11 boxes, 1 reject gate

The encoder's pooled output drives the intent head, while a side decoder supplies text only for entities; the reject gate then chooses route, clarify or fallback. Encoders expect 16 kHz audio, so 8 kHz phone audio is upsampled first and the model still sees only telephone-band content. The cascaded baseline (Saaras transcript, then a text classifier) is not drawn here: it runs on the same test sets and is built first.

Data collection (before Day 1, not counted in the 7 days)

Start with a small, reproducible dataset that proves the whole pipeline, and scale up only after the ready gate below passes. Real recordings are the likeliest bottleneck, so they are kept small and test-only.

Step 1: Fix the task

Pick one inbound domain, such as banking or insurance, with about 10 intents: balance, recent transactions, block card, EMI due, report fraud, complaint status, update details, branch timing, pay bill, transfer to agent. Define the "other" class (next section) and the entity slots: amount, date, last four digits, phone number, person name, and product or plan name. Put product names in a gazetteer and hold some back as unseen test entities. Freeze this taxonomy before any audio is made.

Step 2: Generate text

Use an LLM to write about 120 train, 30 dev and 30 test utterances per intent, plus about 600 "other" utterances across the five types.

- **Variation:** Devanagari Hindi, Romanised Hinglish, English-heavy mixes, and one more language if time allows; casual and formal registers, filler words and self-corrections.
- **Cleaning and splitting:** deduplicate by embedding similarity and split by paraphrase cluster, not randomly. Hand-check about 10%.

Step 3: Build the minimum audio set

- **Synthetic training audio:** render with TTS (Sarvam's Bulbul via API, or an open Indic TTS; check terms and credits). About 10 intents x 120 utterances x 2 voices x 2 acoustic conditions gives roughly 4,800 clips, or 4 to 5 hours. Degrade like a phone call: resample to 8 kHz, apply a G.711 mu-law codec, mix in noise at 0 to 20 dB SNR (MUSAN or DEMAND), add reverb, packet loss and clipping. Dev and test use TTS voices never seen in training.
- **Real recordings, test-only:** about 150 scripted utterances, in-scope and "other", from 5 to 8 speakers over phone calls or voice notes, converted to 8 kHz. Get consent, use fake account numbers, and keep every speaker out of any other split.
- **Real non-intent speech:** about 900 clips from IndicVoices (natural, spontaneous speech in 22 languages), split by speaker into about 600 for training the "other" class and 300 for testing.

Step 4: Scale up only after the gate passes

If the baseline and pipeline work, add more voices and paraphrases (toward 250 utterances per intent) and more real recordings (toward 300) in the background. None of this is required for the core result.

Step 5: Fix the evaluation sets before training

Set	Content	Reported in
T1	Clean synthetic, unseen voices and paraphrase clusters, in-scope	Synthetic table
T2	8 kHz noisy synthetic, in-scope and "other"	Synthetic table
T3	Real recordings from unseen speakers, in-scope and "other"	Real-speaker table
T4	Real non-intent speech (IndicVoices, unseen speakers)	Real-speaker table
T5	Held-out entity values	Entity results (stretch)
T6	MASSIVE hi-IN utterances synthesised to audio	Public-benchmark table (stretch)

Ready gate: Day 1 does not start until all of these hold

- [ ] Intent taxonomy, "other" types and entity slots are frozen.
- [ ] Splits by paraphrase cluster, TTS voice and speaker are documented.
- [ ] T1 to T4 exist with fixed file lists (T3 at least 150 clips, T4 at least 300).
- [ ] Licences, terms and recording consent are noted in a dataset card stub.

The other class and false triggers

The "other" class is five kinds of input, each tested separately, because a model can look accurate while still triggering the wrong action.

Type	Example	Data source
Out-of-domain request	A weather or cricket-score question	LLM text, then TTS
Chit-chat and greetings	A greeting or small talk in Hinglish	LLM text, then TTS
Ambiguous in-domain	A vague money-related remark that fits no single intent	LLM text, reviewed by hand, then TTS
Unsupported near-miss	A banking request the system has no intent for	LLM text, then TTS
Non-intent audio	Silence, noise, filler-only speech, another language	Noise clips, IndicVoices, other-language speech

Report in-scope accuracy and, separately, the false-accept rate per type: the share of that type routed to a real intent. Compare models at fixed in-scope recall (for example 90% and 95%), and never report overall accuracy alone.

Models and training

Build in stages, so each one gives a usable result even if the next is cut.

Stage 0: cascaded baseline (Day 1). Run Saaras on every test clip, then classify the transcript with a text intent classifier: a character n-gram logistic regression first, then MuRIL or IndicBERT if time allows. This is the reference for everything else, and the same text model later serves as the teacher for distillation.

Stage 1: probe frozen encoders (Day 2). Cache per-layer mean and standard-deviation pooled embeddings once, then train a logistic regression or small MLP on them. Compare XLS-R (300M), the Whisper-small encoder and IndicWav2Vec (confirm availability), plot accuracy against layer depth, and choose the encoder and layers to fine-tune.

Stage 2: fine-tune the best encoder (Day 3). Add attentive statistics pooling and a small MLP head over the intents plus "other". Freeze the lower layers, unfreeze the top few with a lower learning rate, and use SpecAugment, class weights and mixed precision. Early-stop on a dev set of unseen voices and speakers. Compare with the baseline on T1 to T4.

Stage 3: multi-task (Day 5, stretch). Add a CTC loss over characters on the synthetic text and a distillation loss from the text teacher, and measure what each adds.

Guarding against TTS artifacts. Use many voices, speed and pitch perturbation, the full degradation chain, dev sets with unseen voices, and report the real-speaker table prominently.

Compute. Stage 1 runs on a CPU or a free GPU. Stages 2 and 3 fit a free Colab or Kaggle GPU when only the top layers are fine-tuned; keep a smaller encoder as the fallback.

Reject gate and entity handling

The reject gate decides whether the model's intent is safe to act on; entity handling covers values never seen in training.

Reject gate (Day 4, core).

- **Training signal:** the "other" class, fed with synthetic "other" audio and the IndicVoices training split.
- **Signals:** max probability, margin, entropy, an energy score, the distance from the pooled embedding to each intent's centroid, and agreement of the predicted intent across 3 to 5 noisy copies of the same audio.
- **Risk model:** a small logistic regression or gradient-boosted model over these signals, calibrated with temperature scaling or isotonic regression. Choose thresholds on validation data only, then report on the test sets.
- **Yardstick:** max-probability thresholding, compared at the same in-scope recall and broken down by type of "other" input.

Entities (Day 5, stretch).

- Decode the auxiliary CTC head's character transcript only for utterances routed to an intent that needs entities.
- Parse digits, Hindi number words, amounts and dates with rules, and match names and products to the gazetteer in phonetic or transliterated space.
- A span with no match is flagged as unknown and triggers a clarification instead of a guess.
- If the auxiliary decoder is too weak, fall back to Saaras for entity spans only, and say so in the report.

Evaluation and reporting rules

Evaluate on speaker-disjoint and voice-disjoint splits only, with bootstrap confidence intervals, and report failures as readily as wins.

Core comparisons (must-have)

1. Baseline, frozen-encoder probe and fine-tuned model, on T1 to T4.
2. Reject methods: max-probability against the risk model, at fixed in-scope recall, per type of "other" input.
3. Latency: real-time factor on CPU and GPU, and milliseconds from end of speech to decision, for the baseline and the end-to-end model.

Stretch comparisons

1. Encoder choice and layer analysis beyond the Day 2 probe.
2. CTC loss, distillation, and both together.
3. Synthetic-only training against synthetic plus real data; a data-scale curve.
4. Accuracy against SNR and by language mix.
5. Entity accuracy on T5, with and without phonetic matching.
6. The MASSIVE-derived test T6.

Reporting rules

- Three result tables, never merged: synthetic (T1, T2), real speakers (T3, T4), public benchmark (T6).
- T6 is a controlled test on synthesised MASSIVE text, not natural telephony, and is labelled that way.
- Thresholds are chosen on dev data, never on the test sets.
- If the end-to-end model does not beat the cascade, say so and explain why.

Seven-day plan

The core result is Days 1 to 4 plus the report on Day 7. Days 5 and 6 are partly stretch.

Day	Work	Output
Before Day 1	Data collection and the ready gate (not counted)	T1 to T4 fixed, dataset card stub

Day	Work	Output
1	Evaluation harness and metrics; cascaded baseline on T1 to T4; text teacher	Baseline results
2	Cache per-layer features; probe XLS-R, Whisper-small encoder and IndicWav2Vec; pick the encoder	Probe results, layer curves
3	Fine-tune the top layers with the "other" class	Fine-tuned model against baseline on T1 to T4
4	Reject gate: max-probability against the risk model; calibration; false accepts per "other" type	Operating-point results
5	Stretch: CTC and distillation ablation; entity matcher and T5	Ablation and entity results
6	Error analysis, latency, remaining ablations; T6 as its own table; freeze the results tables	Final numbers
7	Report, README, dataset card, model card, optional demo	Submission

Cut line. If time slips, drop in this order: the optional demo, T6, entities, the CTC and distillation ablation. Day 3 is a decision point: if fine-tuning cannot beat the frozen probe, spend Day 5 on error analysis instead of multi-task training.

Risks and assumptions

The main risks are scope and data, not modelling.

- **Scope and deadline unconfirmed:** the plan assumes Problem 1 alone and a 7-day core. If Meenakshi asks for both problems, the second (semantic VAD) becomes a separate effort.
- **Data collection overruns:** keep the dataset small, enforce the ready gate, and scale up only afterwards.
- **TTS artifacts:** many voices, heavy augmentation, unseen-voice dev sets, and a prominent real-speaker table.

- **End-to-end loses to the cascade:** report it honestly. The comparison, the error analysis and the reject-gate results are still the contribution.
- **Over-claiming Indic generalisation:** keep the three result tables separate and word claims to match the data behind them.
- **Small real set:** test-only, speaker-disjoint, with confidence intervals, and described as a sanity check, not a benchmark.
- **Weak entity decoding:** fall back to Saaras for entity spans only, and say so.
- **Compute, API cost, licences, consent:** cache features, fine-tune only top layers, start small, check TTS and dataset terms, and get consent for every recording.

Deliverables

- A GitHub repository with data-generation scripts, training code, notebooks, results and a clear README.
- A 5 to 6 page report with limitations stated first, the three separate results tables, and the per-type false-accept results.
- A dataset card and a short model card.
- Optional: a small Gradio demo that takes a recording or file and shows the intent, the risk score and the decision.

Literature and sources to verify

Several of these were recalled from memory, so confirm titles, authors and years before citing them in the report.

- **End-to-end spoken language understanding:** Lugosch et al. 2019, speech model pre-training for end-to-end SLU (Fluent Speech Commands); Lugosch et al. 2020, using speech synthesis to train end-to-end SLU models, which is the closest precedent for training on TTS audio; SLURP (Bastianelli et al. 2020).
- **Speech encoders and benchmarks:** wav2vec 2.0 (Baevski et al. 2020), XLS-R (Babu et al. 2021), MMS (Pratap et al. 2023), Whisper (Radford et al. 2022), and the SUPERB benchmark, whose intent-classification task evaluates frozen encoders with learned layer weights.
- **Indic speech resources:** Vistaar and IndicWhisper (github.com/AI4Bharat/vistaar), IndicVoices (huggingface.co/datasets/ai4bharat/IndicVoices), and AI4Bharat's IndicWav2Vec and IndicConformer models.

- **Intent data:** MASSIVE (huggingface.co/datasets/AmazonScience/massive) and CLINC150 for out-of-scope evaluation (Larson et al. 2019).
- **Training and augmentation:** SpecAugment (Park et al. 2019), attentive statistics pooling (Okabe et al. 2018), and MUSAN and DEMAND noise sets.
- **Rejection and calibration:** the max-softmax baseline (Hendrycks and Gimpel 2017), Mahalanobis distance (Lee et al. 2018), energy-based scores (Liu et al. 2020), and Guo et al. 2017 on calibration.
- **Rare-entity biasing, for the entity discussion:** Deep Context (Pundak et al. 2018) and TCPGen (Sun et al. 2021).